

DevOps Performance & Monitoring Guide

Doc Ref: ACQO-DOC-006 | v1.0 | 2025

Autonomous Code Quality & Optimization Intelligence System

1. Overview

Observability and performance monitoring are essential to detecting regressions, identifying bottlenecks, and validating optimization outcomes in production environments. This guide defines monitoring standards, SLO thresholds, alerting rules, and performance regression detection criteria for all services monitored by the ACQO system.

2. Observability Principles (Three Pillars)

Pillar	Definition	Tooling	Key Signal
Metrics	Numeric measurements of system state over time	Prometheus + Grafana	Response time, error rate, CPU, memory
Logs	Structured event records with context and timestamps	ELK Stack / Loki	Error messages, trace IDs, business events
Traces	Distributed request path tracking across service boundaries	Jaeger / OpenTelemetry	Latency per span, cross-service dependencies

3. Application Performance Monitoring (APM)

All ACQO-monitored services must emit the following standard APM signals:

Metric	Description	Unit	Collection Method
avg_response_time	Mean latency per request over 5-min window	ms	APM agent / middleware timer
p95_response_time	95th percentile latency — true user experience	ms	Histogram bucket
peak_response_time	Maximum observed latency in recording window	ms	APM max aggregation
error_rate	Proportion of requests returning 5xx errors	%	Request counter / error counter
throughput	Requests successfully processed per second	req/s	Counter + rate calculation
saturation	CPU and memory utilization per pod/container	%	cAdvisor / node exporter

4. SLO Thresholds & Alert Rules

Service	Avg Response SLO	Error Rate SLO	Throughput Min	Current Status	Alert Triggered?
payment-service	< 300ms	< 1%	> 250 req/s	220ms / 0.8%	No

Service	Avg Response SLO	Error Rate SLO	Throughput Min	Current Status	Alert Triggered?
analytics-engine	< 400ms	< 1%	> 100 req/s	480ms / 1.2%	YES — Both
auth-service	< 200ms	< 0.5%	> 300 req/s	180ms / 0.4%	No
dashboard-api	< 350ms	< 1%	> 200 req/s	260ms / 0.6%	No

■ **Active Alert:** analytics-engine is breaching both avg response time (480ms vs 400ms SLO) and error rate (1.2% vs 1% SLO). Escalation to Data Team on-call triggered automatically.

5. Error Rate Thresholds & Severity Classification

Error Rate	Severity	Alert Channel	Response SLA	Automated Action
< 0.5%	Normal	Dashboard only	No SLA	None
0.5–1.0%	Warning	Slack #alerts channel	Within 4 hrs	Auto-ticket created
1.0–3.0%	Critical	PagerDuty (P2)	Within 1 hr	On-call page + ACQO flag
> 3.0%	Severe	PagerDuty (P1)	Within 15 min	Incident declared + rollback consideration

6. System Latency Monitoring

- **P50 / P95 / P99 Tracking:** Median latency hides tail latency issues. Always monitor P95 and P99 percentiles for user-facing services.
- **Latency Budget Allocation:** For a 300ms end-to-end SLO, allocate budgets: Gateway (20ms), Auth (30ms), Business Service (180ms), Database (60ms), Response serialisation (10ms).
- **Downstream Dependency Tracing:** Use distributed tracing to identify which downstream call (DB, cache, external API) is contributing to latency spikes.
- **Cold Start Monitoring:** Container cold starts can spike P99 latency. Monitor startup time metrics for auto-scaled services.

7. Performance Regression Detection

ACQO detects performance regressions by comparing rolling baselines against new deployments:

Detection Method	Trigger Condition	Comparison Window	Action
Baseline comparison	Avg latency increases >15% post-deploy	30-min rolling baseline	Auto-flag deployment
Error rate spike	Error rate doubles within 10 min of deploy	10-min post-deploy window	Rollback recommendation
Throughput drop	Throughput falls >20% without traffic change	15-min window	Agent investigation triggered
P99 breach	P99 exceeds 3x the P50 baseline	Continuous	Performance report generated
Memory leak detection	Heap usage grows > 5% per hour continuously	1-hr trend	Restart + investigation flag

Monitoring & Alerting Workflow



✓ Recommended Practices	✗ Prohibited Practices
✓ Monitor P95/P99 latency, not just average response time	✗ Disable verbose logging to improve response time — fix the code instead
✓ Define SLOs per service before deploying to production	✗ Suppress alerts during incidents without root cause documentation
✓ Use distributed tracing to pinpoint latency contributors	✗ Use average response time as the sole performance indicator